WEBRUM-04: Session Analysis

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 4 of 10 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

User session analysis is the foundation of understanding how real users interact with your web applications. A session represents a complete visit — from the first page load to the last interaction before timeout. By analyzing sessions, you can identify user journey patterns, measure engagement, track conversions, calculate bounce rates, and understand how geographic location and device type impact the user experience.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with session capture active
Session Properties	Custom session properties configured (optional but recommended)
Permissions	`storage:events:read`
Previous Notebook	WEBRUM-01: RUM Fundamentals

1. Session Properties

Dynatrace automatically captures standard session properties and allows you to define custom ones:

Standard Properties

Property	Description	Example Values
`sessionId`	Unique session identifier	`abc123def456`
`userType`	Session classification	`REAL_USER`, `ROBOT`, `SYNTHETIC`
`application`	Application name	`MyWebApp`
`duration`	Total session duration	`300000000000` (nanoseconds)
`userActionCount`	Number of user actions	`15`
`totalErrorCount`	Number of errors in session	`2`
`country`	User's country	`United States`
`city`	User's city	`Chicago`
`osFamily`	Operating system	`Windows`, `macOS`, `iOS`

Property	Description	Example Values
`browserFamily`	Browser	`Chrome`, `Firefox`, `Safari`
`screen.width`	Screen width in pixels	`1920`
`screen.height`	Screen height in pixels	`1080`
`connection.type`	Network connection type	`4g`, `wifi`, `ethernet`

Custom Session Properties

Define custom properties via **Settings > Web and mobile monitoring > Session and user action properties**. Common examples:

- `session.user_id` — Authenticated user identifier
- `session.plan_type` — Subscription tier (free, pro, enterprise)
- `session.cart_value` — Shopping cart total
- `session.ab_variant` — A/B test variant

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; primary_tags.application
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.waiting_duration
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the "real_user" literal itself could NOT be confirmed here and is the
documented value form;
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Explore session data — view available fields
fetch user.sessions, from:-1h
| filter dt.rum.user_type == "real_user"
| limit 5
```

2. Session Segmentation

Segmenting sessions helps identify patterns across different user groups. Common segmentation dimensions include engagement level, error impact, and return frequency.

```
// Engagement segmentation — bucket sessions by number of actions
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| fieldsAdd engagement = if(user_action_count == 1, "Bounce",
    else: if(user_action_count &lt;= 3, "Low (2-3 actions)",
    else: if(user_action_count &lt;= 10, "Medium (4-10 actions)",
    else: "High (10+ actions)")))
| summarize session_count = count(),
    avg_duration = avg(duration),
    avg_errors = avg(error.count),
    by:{engagement}
| sort session_count desc
```

```
// Error-impacted sessions — how many sessions had errors?
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize total_sessions = count(),
    error_sessions = countIf(error.count &gt; 0),
    by:{primary_tags.application}
| fieldsAdd error_session_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort error_session_pct desc
```

3. User Journey Mapping

Understanding the sequence of actions within sessions reveals common navigation paths, dead ends, and drop-off points.

```
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**,
but was written
// against names that are null on New RUM data, so these cells returned
nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.classifier ==
"page_summary"
//   action.type                       -&gt; user_action.type
(hard_navigation | same_view)
//   action.name                       -&gt; page.detected_name
//   web_vitals.largest_contentful_paint-&gt; lcp.start_time       (327,099
populated)
//   web_vitals.cumulative_layout_shift -&gt; cls.value            (387,254
populated)
//   app.name                          -&gt; dt.rum.application.id
// UNITS CHANGE WITH THE FIELD. web_vitals.* was a nanosecond DURATION, so `/
1ms` was correct for
// it; lcp.start_time is a PLAIN NUMBER already in milliseconds, and dividing
it by 1ms yields
// null. Compare it against the 2500/4000 ms thresholds directly.
// `web_vitals.*` does still exist in the same schema, but carried 16 records
in 30 days against
// lcp.*'s 327,099 — it is not a different RUM generation, just a rarely-
populated sibling.
// Top entry pages — where do users start their journey?
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| summarize first_action = takeFirst(page.detected_name), by:
{dt.rum.session.id}
| summarize entry_count = count(), by:{first_action}
| sort entry_count desc
| limit 10
```

```
// Top exit pages — where do users leave?
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| summarize last_action = takeLast(page.detected_name), by:{dt.rum.session.id}
| summarize exit_count = count(), by:{last_action}
| sort exit_count desc
| limit 10
```

```
// Session duration distribution by hour of day — when are users most active?
fetch user.sessions, from:-7d
| filter dt.rum.user_type == "real_user"
| fieldsAdd hour = getHour(timestamp)
| summarize session_count = count(),
    avg_duration_sec = avg(duration / 1s),
    by:{hour}
| sort hour asc
```

4. Conversion Tracking

Conversion tracking identifies which sessions completed a desired business action (purchase, signup, form submission). This requires either:

- **Session properties** marking conversion events
- **Specific user actions** that indicate conversion (e.g., a "Thank You" page load)

Conversion via Action Name Detection

If your conversion page has a recognizable URL pattern, you can detect conversions from user actions:

```
// Conversion rate — sessions that reached a checkout/confirmation page
// Adapt the filter to match your conversion page URL pattern
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize total_sessions = count(),
    by:{primary_tags.application}
| append [
    fetch user.events, from:-24h
    | filter characteristics.classifier == "page_summary"
    | filter contains(page.detected_name, "confirmation") or
contains(page.detected_name, "thank-you") or contains(page.detected_name,
"checkout-success")
    | summarize converted_sessions = countDistinct(dt.rum.session.id), by:
{primary_tags.application}
  ]
```

> **Tip:** For accurate conversion tracking, use Dynatrace session properties to flag conversion events. This avoids reliance on URL pattern matching, which can be

> fragile.

5. Bounce Rate Analysis

A "bounce" is a session with only one user action — the user loaded one page and left. High bounce rates may indicate poor landing page relevance, slow performance, or broken functionality.

```
// Bounce rate by application
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize total_sessions = count(),
    bounced_sessions = countIf(user_action_count == 1),
    by:{primary_tags.application}
| fieldsAdd bounce_rate_pct = round(toDouble(bounced_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort bounce_rate_pct desc
```

```
// Bounce rate trend over 7 days — track improvement over time
fetch user.sessions, from:-7d
| filter dt.rum.user_type == "real_user"
| fieldsAdd is_bounce = if(user_action_count == 1, 1, else: 0)
| makeTimeseries total = count(), bounces = sum(is_bounce), interval:1d
| fieldsAdd bounce_rate = arrayAvg(bounces) / arrayAvg(total) * 100.0
```

6. Geographic Analysis

RUM data includes geographic information derived from the user's IP address. This helps identify regional performance differences and target optimization efforts.

```
// Sessions by country — top 15 countries by volume
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(geo.country.name)
| summarize session_count = count(),
    avg_actions = avg(user_action_count),
    avg_errors = avg(error.count),
    avg_duration_sec = avg(duration / 1s),
    by:{geo.country.name}
| sort session_count desc
| limit 15
```

```
// Top 10 cities by session count with average performance
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(geo.city.name) and isNotNull(geo.country.name)
| summarize session_count = count(), by:{geo.city.name, geo.country.name}
| sort session_count desc
| limit 10
```

7. Device and Browser Analysis

Understanding the device and browser mix helps prioritize testing and optimization efforts.

```
// Session distribution by browser
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(browser.name)
| summarize session_count = count(),
    avg_errors = avg(error.count),
    by:{browser.name}
| sort session_count desc
| limit 10
```

```
// Session distribution by operating system
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(os.name)
| summarize session_count = count(),
    avg_duration_sec = avg(duration / 1s),
    by:{os.name}
| sort session_count desc
```

```
// Screen resolution distribution — identify common viewport sizes
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(browser.window.width) and isNotNull(browser.window.height)
| fieldsAdd resolution = concat(toString(browser.window.width), "x",
toString(browser.window.height))
| summarize session_count = count(), by:{resolution}
| sort session_count desc
| limit 10
```

8. Summary and Next Steps

In this notebook, we covered:

- **Session properties** — Standard and custom properties for segmentation
- **Engagement segmentation** — Bucketing sessions by interaction depth
- **User journey mapping** — Entry/exit pages and activity by time of day
- **Conversion tracking** — Identifying sessions that completed business goals
- **Bounce rate analysis** — Measuring and trending single-action sessions
- **Geographic analysis** — Session distribution by country and city
- **Device/browser analysis** — Understanding the technology mix of your users

Next Steps

- **WEBRUM-05: Error Analysis** — JavaScript error tracking and impact analysis
- **WEBRUM-06: Performance Analysis** — Deep dive into page load waterfall timings

References

- Define user action and session properties (DT docs)
- USQL — custom session queries (DT docs)
